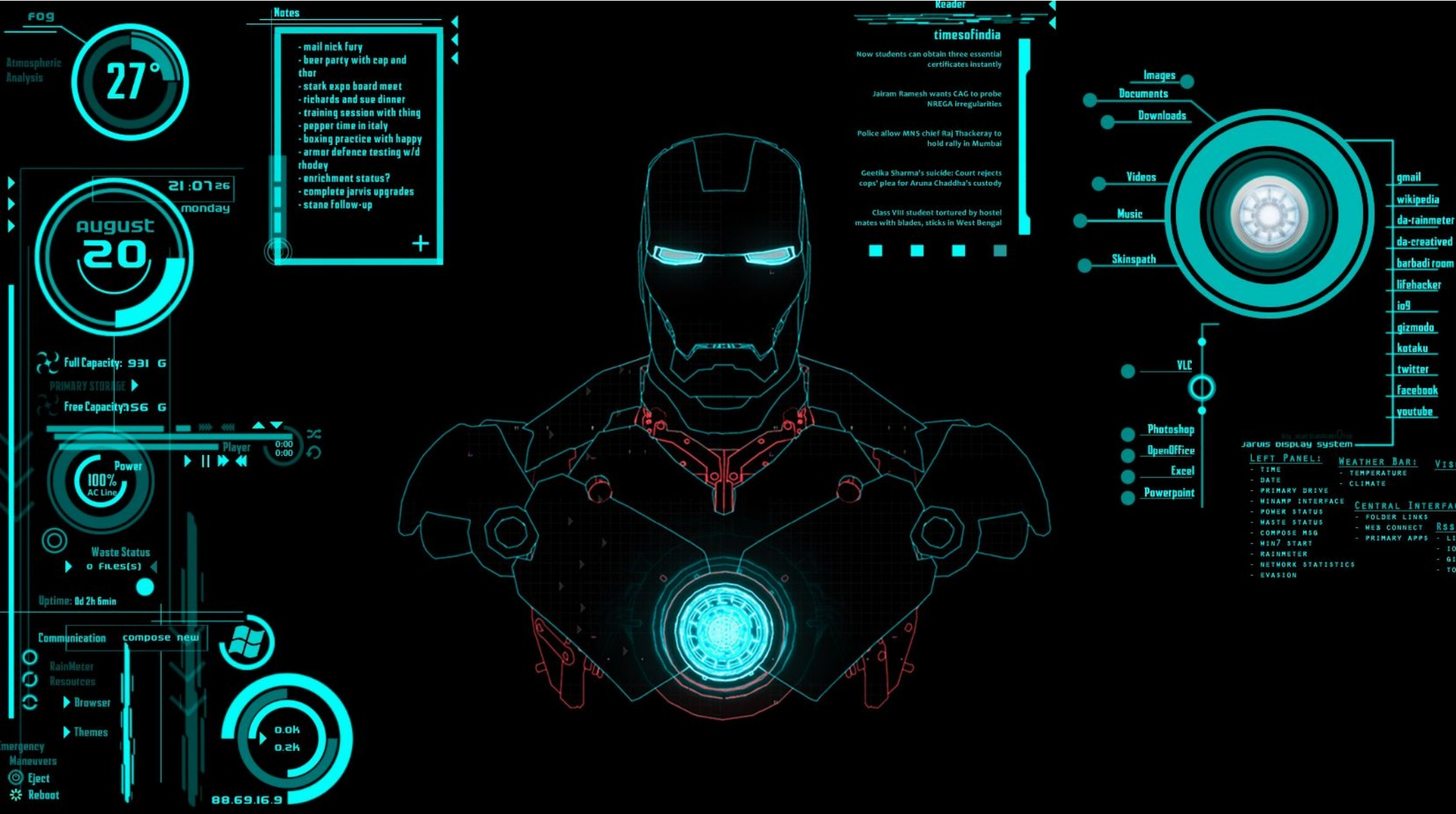




09. Introdução à OpenGL

Prof. Alexandre Krohn

Roteiro

- Introdução a OpenGL
- Características do OpenGL
- Um exemplo em OpenGL
- Setup do ambiente de desenvolvimento

OpenGL

- O OpenGL foi criado em **1992** como uma alternativa aberta e reproduzível ao **Íris GL**, que era a API gráfica proprietária nas estações de trabalho da *Silicon Graphics*.

API OpenGL?

- OpenGL é considerada por muitos uma API (Interface de programação de aplicações) que nos fornece um grande conjunto de funções para manipular gráficos e imagens.

Especificação OpenGL

- No entanto, OpenGL por si só não é um API, mas apenas uma especificação, desenvolvida e mantida pelo **Grupo Khronos**.
- Uma especificação diz quais funções devem existir, quais parâmetros devem receber, quais os retornos e quais comportamentos devem ocorrer quando as funções forem invocadas.

Grupo Khronos

- O Grupo Khronos é um consórcio aberto de mais de **150 empresas** líderes de hardware e software que criam **padrões de aceleração avançados e livres de royalties para gráficos 3D**, realidade virtual e aumentada, visão e aprendizado de máquina.

Grupo Khronos : Empresas

K H R O N O S
GROUP

Over 150 members worldwide
Any company is welcome to join



IKEA Communications AB



Grupo Khronos : Padrões

3D Graphics
Desktop, Mobile, Web
Embedded and Safety Critical



3D Assets
Authoring
and Delivery



Portable XR
Augmented and
Virtual Reality



**Parallel
Computation**
Vision, Inferencing,
Machine Learning



KHRONOS®
SAFETY CRITICAL
ADVISORY FORUM



Guidelines for creating APIs to streamline system safety certification

OpenGL não é

- Uma biblioteca
- Uma game engine
- Um framework
- OpenGL é uma especificação

OpenGL não é

- Não é uma linguagem de programação
- Não tem comandos para criação e gerenciamento de janelas ou para tratar entradas do usuário
- Não tem comandos de alto-nível para descrição de objetos 3D

Por que usar OpenGL

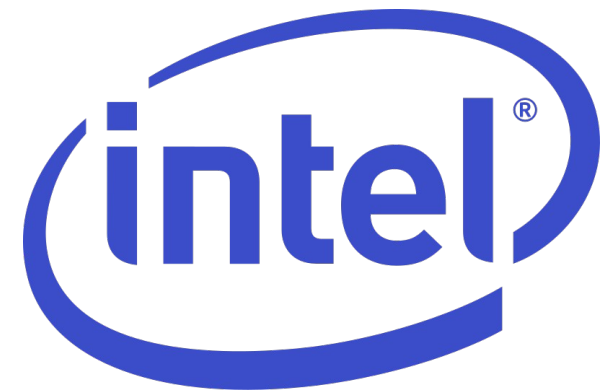
- Programas feitos usando **OpenGL executam suas rotinas gráficas na GPU** (Graphics Processing Unit) ao invés da CPU (Unidade Central de Processamento)
- As GPUs conseguem fazer muitos cálculos em paralelo, e por isso a execução é muito mais rápida.
- Isso sem falar na memória gráfica...

Instalação

- A OpenGL não é instalada pelo desenvolvedor para programar
- Ela já está instalada no computador, pois é parte dos drivers da placa de vídeo
- No entanto teremos que instalar as bibliotecas que nos darão acesso ao OpenGL

Placas de Vídeo

- A maioria dos computadores atuais utiliza uma placa de vídeo dos seguintes fornecedores:



OpenGL é Open-Source?

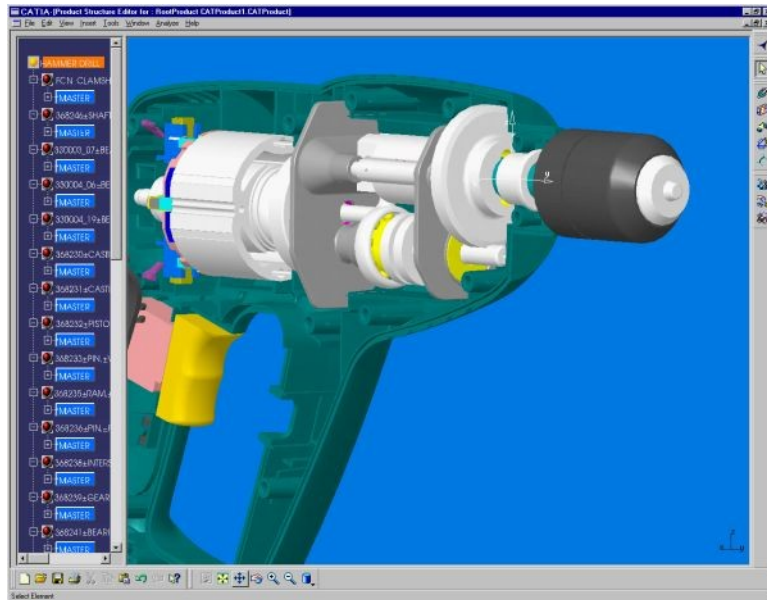
- Não. OpenGL é apenas uma especificação
- Cada fornecedor de dispositivo gráfico a implementa da forma que achar mais conveniente, de forma proprietária

OpenGL é Cross-platform?

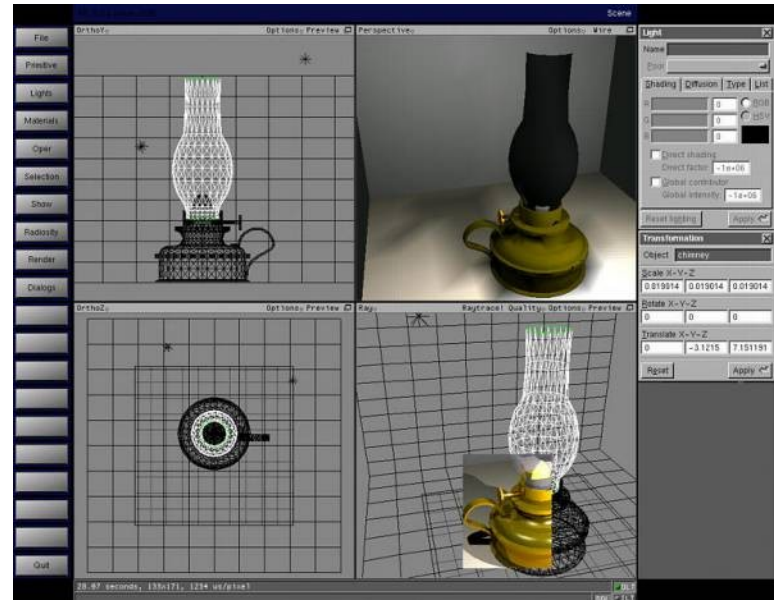
- Não e Sim. Como OpenGL é apenas uma especificação,
- Os fornecedores de dispositivos gráficos podem possuir implementações diferentes para cada plataforma

O que é feito com OpenGL?

Ferramentas CAD



IBM/Dassault CATIA



Moonlight Creator (GPL)

O que é feito com OpenGL?

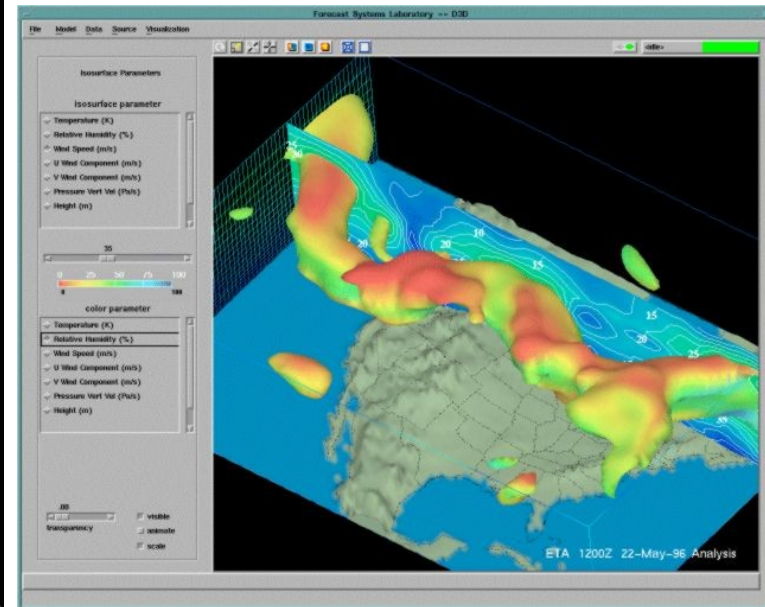
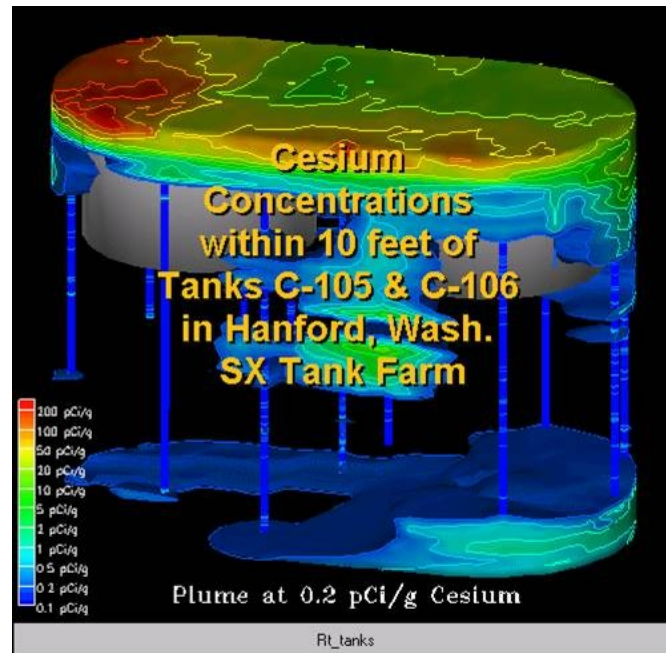
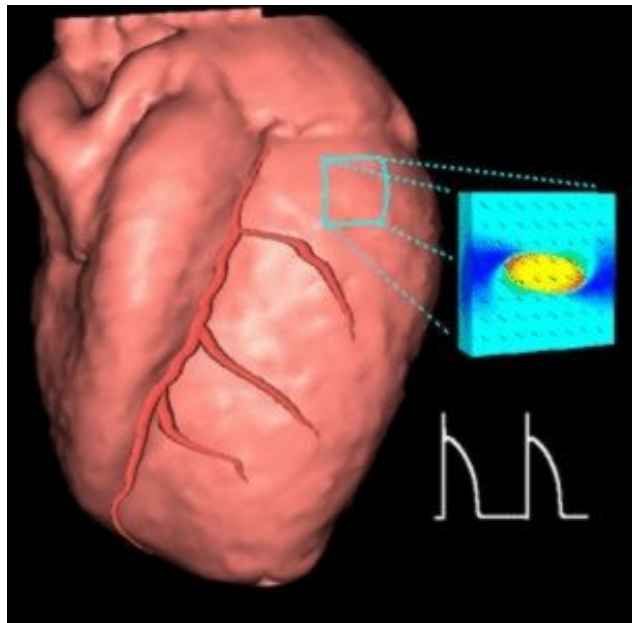
Modelagem e desenho



NewTek LightWave 3D

O que é feito com OpenGL?

Visualização e Simulação



O que é feito com OpenGL?

Jogos



Animações



Histórico

- OpenGL (Criado em 1992)
- OpenGL 2.0
 - 3DLabs
 - OpenGL Shading Language
- OpenGL 2.1
 - OpenGL Shading Language revision 1.20
 - sRGB texture formats
- OpenGL 3.0
 - Criação de objetos assíncrona
- OpenGL 3.2
 - Profiles core e compatibility
- OpenGL 4.0
 - Tesselação e Shading Language 4.0
- OpenGL 4.6 (2017)
 - Filtros anisotrópicos e definição de shaders através de SPIR-V

Como usar OpenGL?

- Por se tratar de uma especificação, é necessário “linkar” as funções dos drivers da placa de vídeo.
 - Isso é feito com bibliotecas auxiliares, como **GLEW** ou **GLAD**
- Por não ter suporte a janelas do SO, é necessário usar alguma biblioteca para interagir com o sistema
 - Pode ser feito com **GLUT** ou **GLFW**

Bibliotecas auxiliares

- Utilizaremos:
 - **GLEW** (*The OpenGL Extension Wrangler Library*)
 - é uma biblioteca de carregamento de extensão C/C++ multiplataforma e open-source. GLEW fornece mecanismos de tempo de execução eficientes para determinar quais extensões OpenGL são suportadas na plataforma de destino.
 - (<http://glew.sourceforge.net/>)
 - **GLFW** (*Graphics Library FrameWork*)
 - é uma biblioteca open-source e multiplataforma para desenvolvimento OpenGL, OpenGL ES e Vulkan no desktop. Ele fornece uma API simples para criar janelas, contextos e superfícies, recebendo entradas e eventos.
 - (<https://www.glfw.org/>)

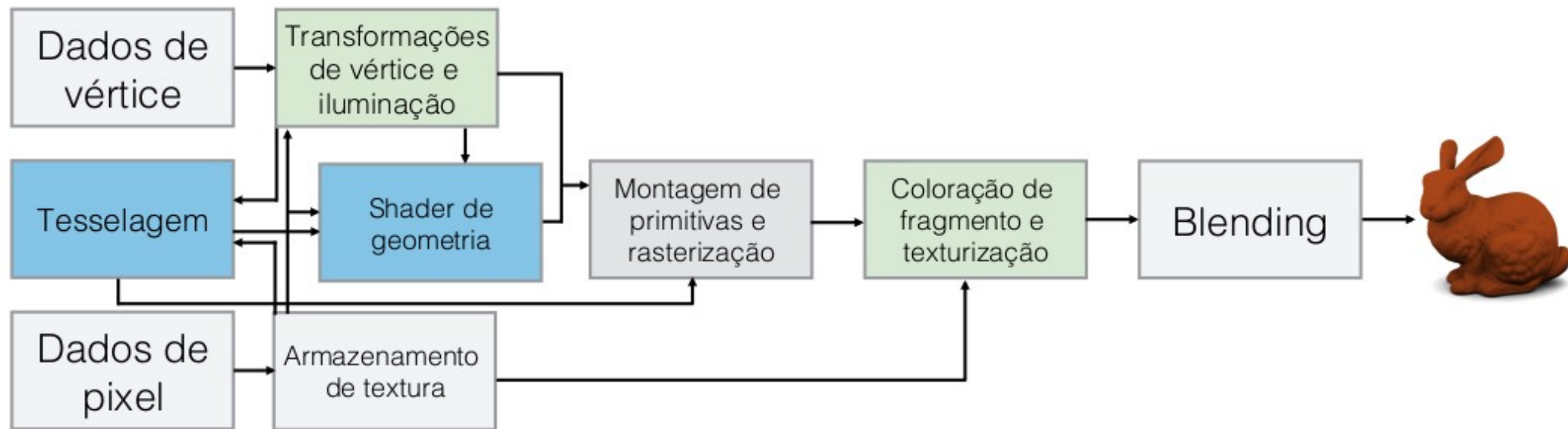
Como usar OpenGL?

- Importa-se as bibliotecas
- Define-se a janela e o contexto
- Definem-se os vértices do objeto a ser desenhado
- Informa-se ao OpenGL o que fazer com os vértices

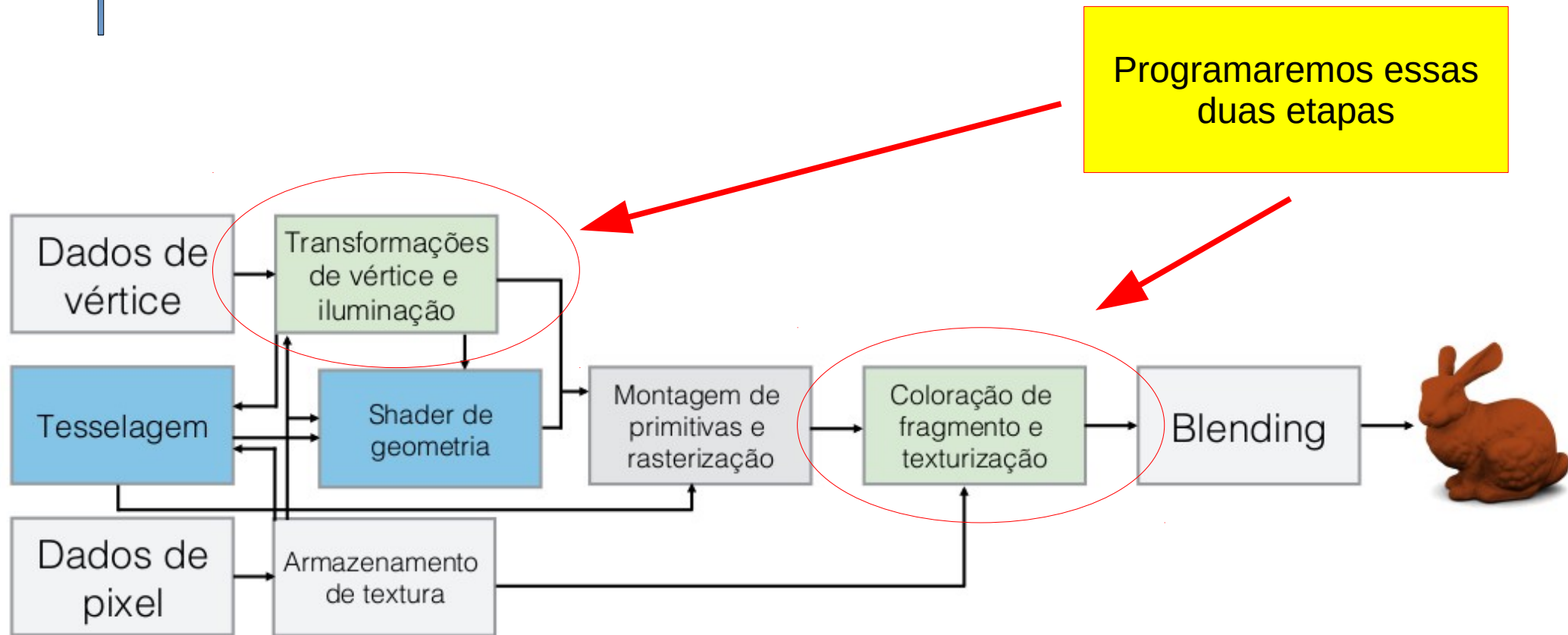
OpenGL : Máquina de estados

- Uma aplicação OpenGL é uma máquina de estados.
 - A cor corrente é uma variável de estado.
 - Há muitas outras!
 - Todas as variáveis de estado possuem valores default
 - O efeito de muitas delas pode ser habilitado e desabilitado

OpenGL : Pipeline de Execução



OpenGL : Pipeline de Execução



Exemplo OpenGL + GLFW

```
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <iostream>
```

Inclui as bibliotecas
necessárias

```
int main0(void)
{
```

```
    GLFWwindow* window;
```

Inicializa a biblioteca
de janelas

```
    if (!glfwInit())
        return -1;
```

```
    window = glfwCreateWindow(640, 480, "GLFW Básico : Legacy OpenGL", NULL,
NULL);
```

```
    if (!window)
```

```
    {
```

```
        glfwTerminate();
```

```
        return -1;
```

```
    }
```

Cria a janela
do aplicativo

```
    glfwMakeContextCurrent(window);
```

Vincula a janela
ao contexto OpenGL

```
    if(glewInit()!=GLEW_OK) {
```

```
        std::cout << "Ocorreu um erro iniciando GLEW!" << std::endl;
```

```
    } else {
```

```
        std::cout << "GLEW OK!" << std::endl;
```

```
        std::cout << glGetString(GL_VERSION) << std::endl;
```

```
    }
```

Exemplo OpenGL + GLFW

```
while (!glfwWindowShouldClose(window))  
{  
  
    glClear(GL_COLOR_BUFFER_BIT);  
  
    glBegin(GL_TRIANGLES);  
        glVertex2f(-0.5f,-0.5f);  
        glVertex2f(0.0f,0.5f);  
        glVertex2f(0.5f,-0.5f);  
    glEnd();  
  
    glfwSwapBuffers(window);  
  
    glfwPollEvents();  
}  
  
glfwTerminate();  
return 0;  
}
```

Loop principal

Limpa janela
(pinta o fundo)

Define os vértices e
Desenha um triângulo

Troca os buffers
de frente e de fundo

Aguarda os eventos

Encerra o contexto

Exemplo OpenGL + GLFW

```
while (!glfwWindowShouldClose(window))  
{
```

```
    glClear(GL_COLOR_BUFFER_BIT);
```

```
    glBegin(GL_TRIANGLES);  
        glVertex2f(-0.5f,-0.5f);  
        glVertex2f(0.0f,0.5f);  
        glVertex2f(0.5f,-0.5f);  
    glEnd();
```

```
    glfwSwapBuffers(window);
```

```
    glfwPollEvents();
```

```
}
```

```
glfwTerminate();
```

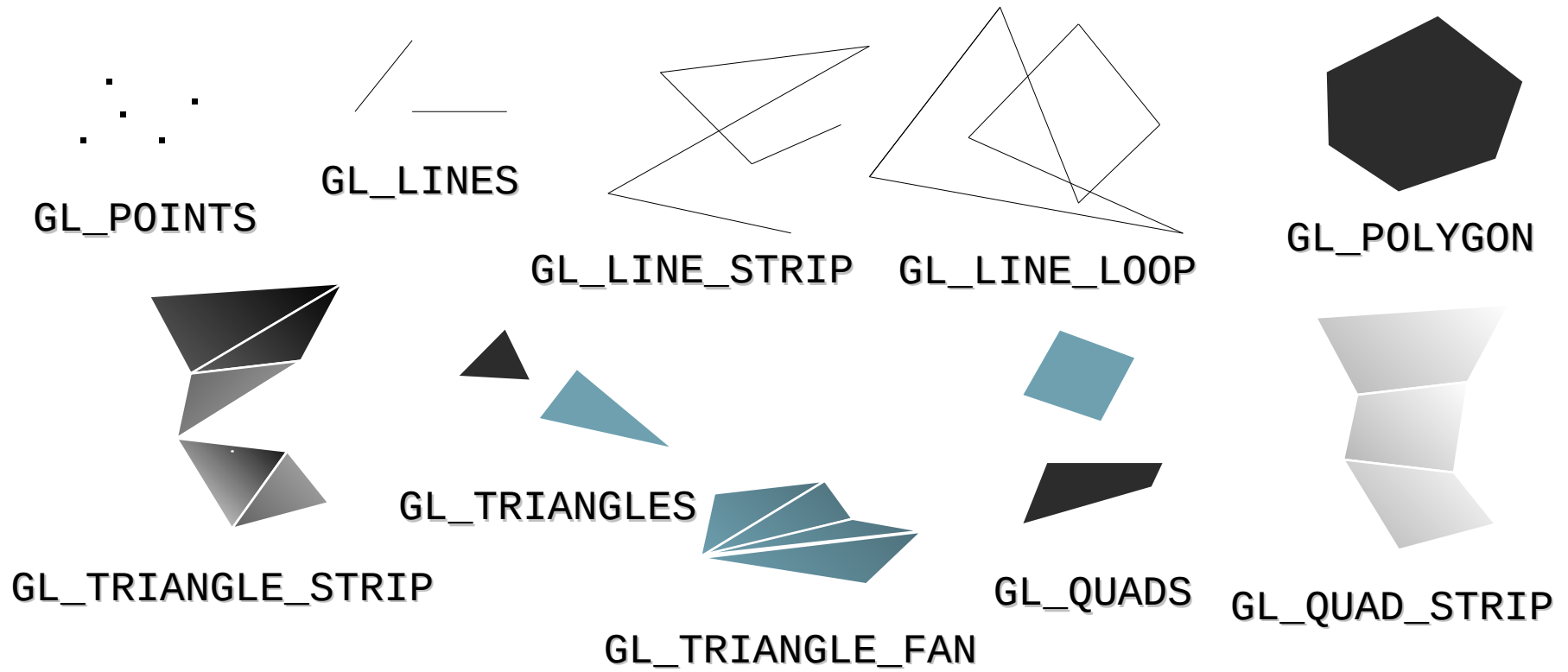
```
return 0;
```

```
}
```

Atenção para área
do programa
que faz o desenho

glBegin(...)

- A função glBegin(...) inicia o desenho de primitivas gráficas a partir dos vértices



glVertex2d(...)

- Os vértices são definidos usando uma família de funções

glVertex3fv(v)

Número de componentes

2 - (x,y)
3 - (x,y,z)
4 - (x,y,z,w)

Tipo de dado

b - byte
ub - unsigned byte
s - short
us - unsigned short
i - int
ui - unsigned int
f - float
d - double

vetor

**omita o "v" qdo
coords dadas uma a uma**

glVertex2f(x, y)

Funções do OpenGL

- De forma geral:

gl{nome da função}{número de argumentos}{tipo dos argumentos}(lista de argumentos);

Exemplos:

– glColor3f => gl Color 3 f

Nome da função

Número de argumentos

Tipo dos argumentos(float)

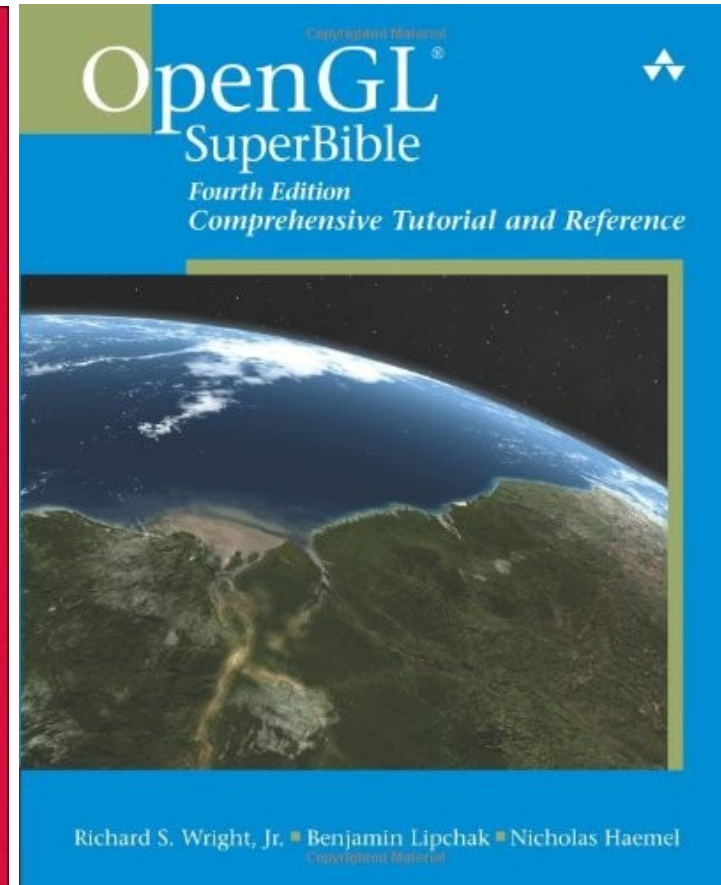
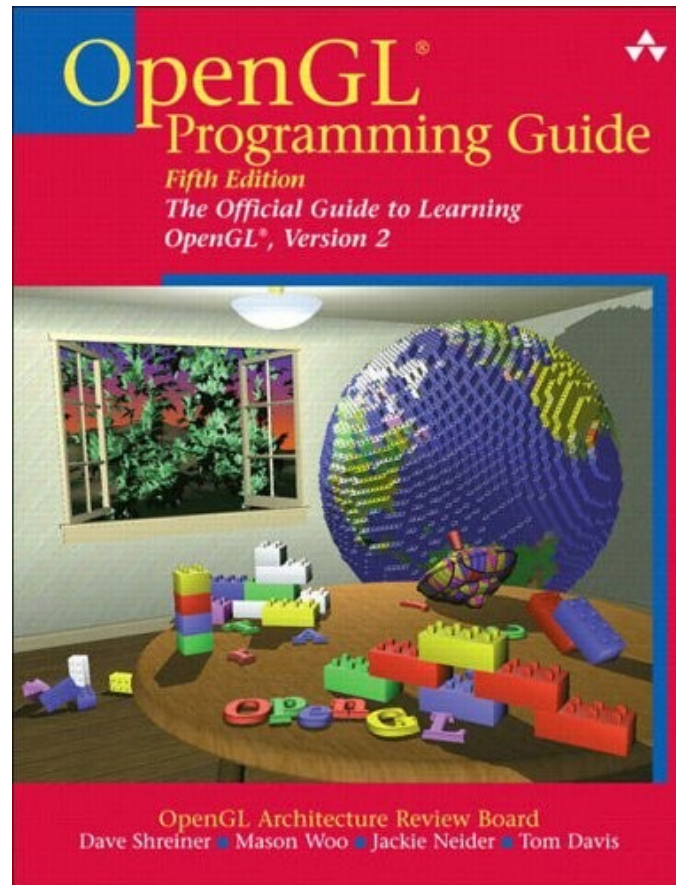
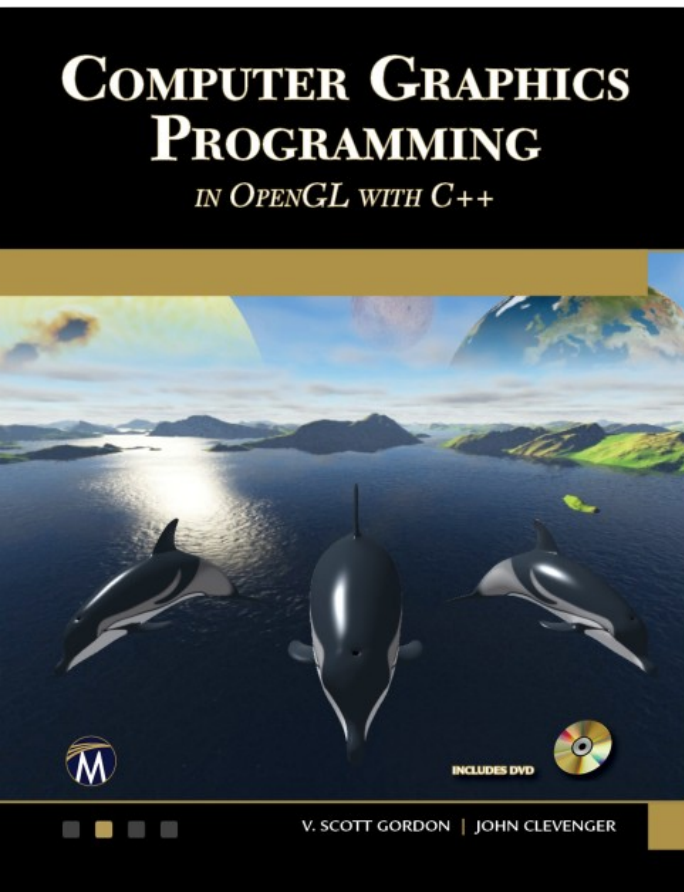
– glLightfv => gl Light f v

Tipo dos argumentos(float vector)

Funções do OpenGL

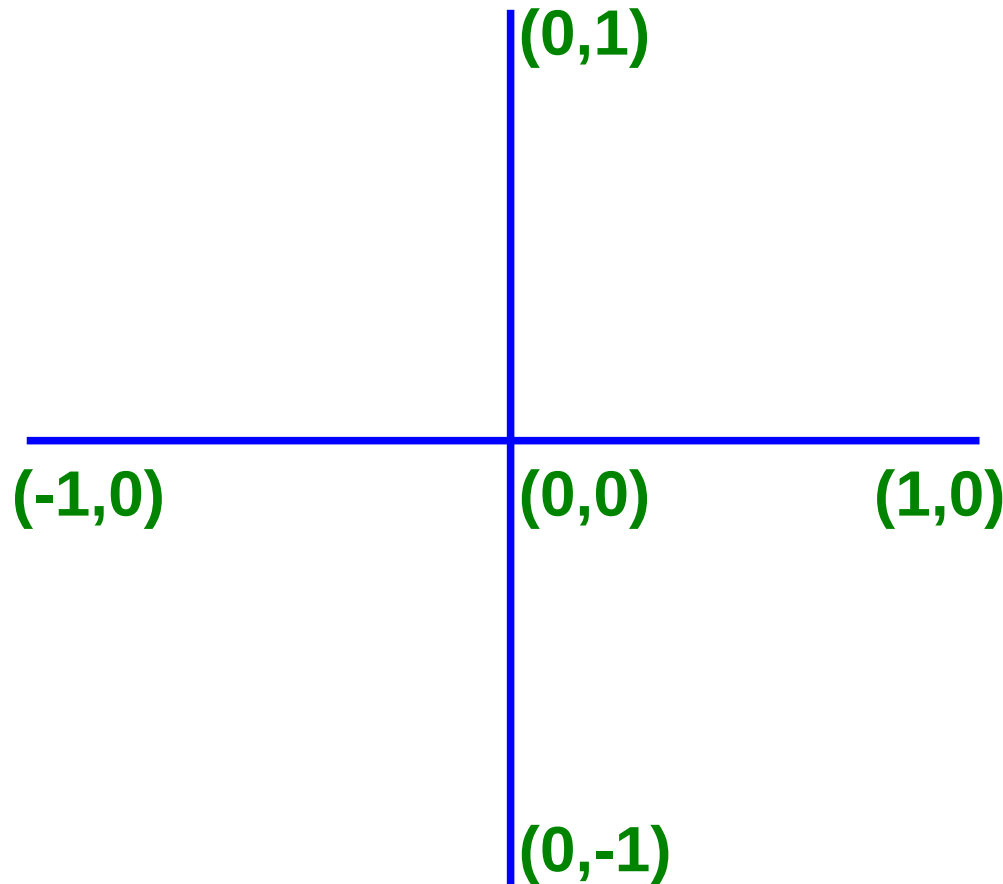
- OpenGL tem cerca de 250 funções
- O uso de livros de OpenGL é muito recomendado
- Documentação também pode ser encontrada em : <http://docs.gl/>

Livros



Coordenadas de tela

- OpenGL utiliza por padrão as coordenadas do SRU entre $(-1, -1)$ e $(1,1)$

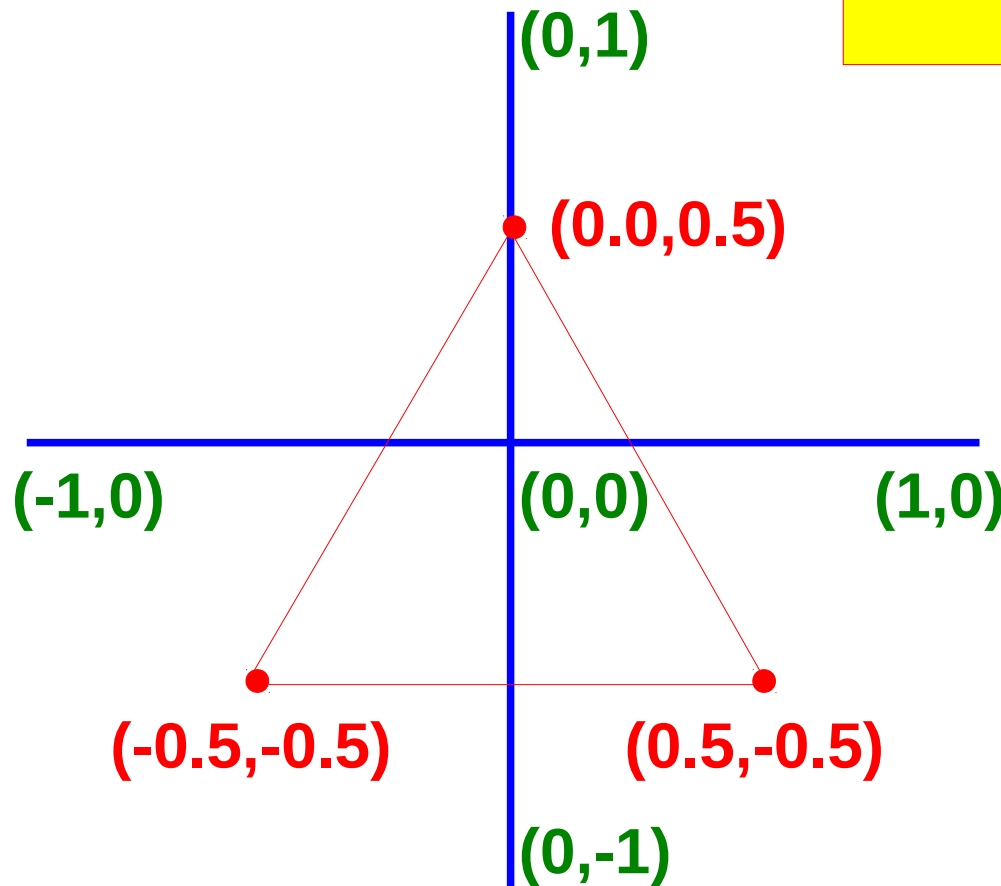


Coordenadas de tela

Os vértices do exemplo de código:

- $(-0.5, -0.5)$
- $(0.0, 0.5)$
- $(0.5, -0.5)$

O mapeamento para o SRD é automático, mas pode ser configurado



OpenGL antiga e atual

- O exemplo demonstrado é útil para testar o setup inicial do ambiente, e ter certeza que tudo está funcionando. É feito em “**Legacy Mode**”
- Na OpenGL corrente, utilizamos **shaders** para informar ao OpenGL como interpretar os vértices.

Dúvidas?



Atividade 1

- Acesse o arquivo ***Setup OpenGL.pdf***, e faça a configuração ali contida no seu ambiente do Code::Blocks para poder desenvolver usando OpenGL

Atividade 2

- Localize as linhas de código a seguir na função `main(..)`

```
glBegin(GL_TRIANGLES);  
    glVertex2f(-0.5f,-0.5f);  
    glVertex2f(0.0f,0.5f);  
    glVertex2f(0.5f,-0.5f);  
glEnd();
```

- Altere a quantidade e/ou os valores dos vértices e troque a primitiva gráfica informada na função `glBegin(..)`. Observe os resultados.